Pipes in the Desert
Prototype Program

E5 - TEAM A

Supervisor:

Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb	OZLVV3	muhammadibrahimshoeb@gmail.com
Arda Gecegörür	EKQNAK	ardagecegorur@gmail.com
Ilgın Tunç	TDQWFF	ilgintunc11@gmail.com
Muhammad Hameez Khan	TFBB32	hameezkhan993@gmail.com
Yahya Akhrikhar	K20UXP	yahyaakhrikhar02@gmail.com
Aasif Mohd	OI6VE6	aasifuk1122@gmail.com

10. Prototype program

10.1 Deployment guide

10.1.1 List of files

File Name	Size	Date	Content
System.java	12.2 KB	02.05.2026	Main game coordinator class; manages session lifecycle, turn order, timer, and end-game logic
Player.java	5.2 KB	02.05.2026	Abstract base class representing a game participant; handles movement and turn state
Plumber.java	7.3 KB	02.05.2026	Extends Player; implements plumber-specific actions: repair, connect, insert pump, pick up components
Saboteur.java	1.9 KB	02.05.2026	Extends Player; implements saboteur-specific actions: puncture pipe, change pump direction
PipeNetwork.java	10.9 KB	02.05.2026	Stores and manages the full network topology; handles adjacency, connection, and pump insertion
NetworkElement.java	4.3 KB	02.05.2026	Abstract base class for all physical network elements; provides shared connectivity behavior
Pipe.java	7.5 KB	02.05.2026	Represents a water-carrying segment; tracks puncture state, free end, occupancy, and water
Pump.java	11.3 KB	02.05.2026	Active routing element; manages input/output selection, breakdown state, and connection limit
Spring.java	3.3 KB	02.05.2026	Water source node; produces water each simulation step
Cistern.java	6.7 KB	02.05.2026	Water destination node; stores delivered water and generates new pipes and pumps

ScoreBoard.java	3.3 KB	02.05.2026	Tracks and compares delivered and leaked water; determines the winning team
GameTimer.java	2.6 KB	02.05.2026	Manages the countdown timer; triggers end-game when time expires
WaterFlowManager.java	6.2 KB	02.05.2026	Evaluates water movement each tick; registers delivered and leaked water
Team.java	2.2 KB	02.05.2026	Groups players of the same role; provides team-level information and member access
Main.java	29.2 KB	02.05.2026	Entry point of the program; displays the welcome message, menu, and launches the game session

10.1.2 Compilation

[The technique, command(s) to be applied to produce the runnable code from the files listed in the previous point. The environment in which the compilation is performed should be conform to the hardware-software system defined in the chapter 2.1 (Requirement definition)]

This section describes the commands required to compile the project source files into runnable Java class files. The compilation environment should conform to the hardware and software system defined in Chapter 2.1, Requirement Definition.

1. Prerequisites

Before compiling and running the project, ensure that the following tools are available on the system.

1.1 Required Software

Java Development Kit (JDK), version 11 or higher

A terminal or command-line environment:
Command Prompt or PowerShell on Windows
Terminal on Linux or macOS

Maven is not required for this project, because the submitted project structure does not include a pom.xml file.

1.2 Verify Java Installation

Open a terminal and run the following commands to confirm that Java is properly installed:

java -version
javac -version

Both commands should report version 11 or higher. If either command is not recognised, install a suitable JDK and ensure that it is added to the system PATH.

2. Extract the Project

Unpack the project archive using the method appropriate for your operating system.

2.1 Windows

Right-click the archive file and select Extract All.
Choose a destination folder and confirm the extraction.

2.2 Linux / macOS

unzip TeamA_Prototype_Submission.zip

3. Navigate to the Project Directory

Open a terminal and change into the extracted project folder:

cd path/to/TeamA_Prototype_Submission

Replace **path/to/TeamA_Prototype_Submission** with the actual path where the archive was extracted.

All subsequent commands must be executed from the project root directory. The project root directory should contain folders such as src, tests, build, scripts, and test-cases.

4. Compile the Project

The Java source files are located under the src/app directory. The compiled class files are placed in the build/classes directory.

4.1 Linux / macOS

Create the output directory and compile the project using:

mkdir -p build/classes
javac -d build/classes -sourcepath src src/app/Main.java

4.2 Windows Command Prompt

Create the output directory and compile the project using:

mkdir build\classes
javac -d build\classes -sourcepath src src\app\Main.java

4.3 Windows PowerShell

Create the output directory and compile the project using:

New-Item -ItemType Directory -Force build/classes
javac -d build/classes -sourcepath src src/app/Main.java

The compilation command compiles Main.java and any other required classes referenced by the program. On successful compilation, the generated .class files will be placed in:

build/classes/app

For example, the following file should exist after compilation:

build/classes/app/Main.class

No errors should appear on successful compilation.

5. Compile Tests

The test source file is located under the tests/app directory.

On Linux or macOS, compile the tests using:

javac -d build/classes -cp build/classes -sourcepath src:tests tests/app/PrototypeTests.java

On Windows Command Prompt or PowerShell, compile the tests using:

javac -d build/classes -cp build/classes -sourcepath "src;tests" tests/app/PrototypeTests.java

After successful compilation, the following file should exist:

build/classes/app/PrototypeTests.class

6. Troubleshooting

If javac is not recognised, the JDK is not installed or is not added to the system PATH. Install JDK 11 or higher and configure the PATH variable correctly.

If Main.java is not found, confirm that you are in the project root directory and that the following path exists:

src/app/Main.java

If app.Main is not found at runtime, ensure that compilation completed successfully and that the correct classpath is being used.

For the main program, check that the following file exists before running:

build/classes/app/Main.class

For the test program, check that the following file exists before running:

build/classes/app/PrototypeTests.class

10.1.3 Run

[The activities, commands, parameters for starting the executable program. The environment should be conform to hardware-software system defined in the chapter 2.1 (Requirement definition).]
w
After successful compilation, the program can be started from the project root directory using the command that matches the chosen build method.

java -cp build/classes app.Main

When the program starts, the user is greeted by the main menu:
Welcome to Pipes in the Desert!

1. Start Game
2. Instructions
3. Exit

The user selects an option by entering the corresponding number from the keyboard.

Main Runtime Behavior

When the user selects the option to start a new game, the system initializes the main components of the prototype. These include the teams, players, pipe network, timer, scoreboard, and water-flow manager. After initialization, the system enters the running game state.

The prototype operates in a turn-based manner. During runtime, the currently active player is identified, and the available actions are presented according to the player's role.

- **Plumber actions** include repairing pipes or pumps, changing pump direction, disconnecting and reconnecting pipes, picking up generated components from cisterns, placing new pipes, and inserting pumps into the network.
- **Saboteur actions** include puncturing pipes and changing pump direction.

The player selects an action through console input. If the selected action depends on a condition, the system asks an interactive yes/no question (e.g., whether to repair or puncture a component). The system validates each action before execution. If the action is valid, the corresponding objects in the network are updated; otherwise, an error message is displayed and the player is asked to choose another action.

After each player action, the system performs a set of automatic updates. These include:

- updating round events,
- generating new pipes or pumps at cisterns when conditions are satisfied,
- randomly breaking pumps when applicable,
- recalculating water flow through the network,
- updating the scoreboard based on delivered and leaked water.

Following these updates, the system advances the turn to the next player and checks the end condition of the game. The end condition is based on the expiration of the game timer.

The game continues in this cycle until the end condition is reached. When the game ends, the system stops the timer, compares the delivered and leaked water values, determines the winning team, and displays the final result to the user.

This runtime behavior enables the prototype to demonstrate the core gameplay scenarios defined in the use cases and supports testing of the system's main functional behavior.

10.2 Test protocols

10.2.1 Test-case 1 -Successful game initialization

Name of the tester	Yahya
Date & time of test	04.05.2026, 14:00
Result	Pass

10.2.2 Test-case 2 -Invalid team distribution

Name of the tester	Yahya
Date & time of test	04.05.2026, 14:00
Result	Pass

10.2.3 Test-case 3 -Valid player movement

Name of the tester	Yahya
Date & time of test	04.05.2026, 14:00
Result	Pass

10.2.4 Test-case 4 -Invalid movement target

Name of the tester	Yahya
Date & time of test	04.05.2026, 14:00

Result	Pass

10.2.5 Test-case 5 -Saboteur punctures pipe

Name of the tester	Yahya
Date & time of test	04.05.2026, 14:00
Result	Pass

10.2.6 Test-case 6 -Plumber repairs pipe

Name of the tester	Aasif
Date & time of test	04.05.2026, 22:00
Result	Pass

10.2.7 Test-case 7 -Plumber repairs pump

Name of the tester	Aasif
Date & time of test	04.05.2026, 22:00
Result	Pass

10.2.8 Test-case 8 -Change pump direction

Name of the tester	Aasif
Date & time of test	04.05.2026, 22:00
Result	Pass

10.2.9 Test-case 9 -Component generation and pickup

Name of the tester	Aasif
Date & time of test	04.05.2026, 22:00
Result	Pass

10.2.10 Test-case 10 -End game and winner declaration

Name of the tester	Aasif
Date & time of test	04.05.2026, 22:00
Result	Pass

10.3 Evaluation

[Evaluation shows every team member's participation rate in the project (percents) ***from the beginning of the project*** *to the date of evaluation. The sum of percent values must be 100.]*

Name of the team member	Participation (%)
Muhammad Ibrahim Shoeb	16.6
Arda Gecegörür	16.6
Ilgın Tunç	16.6
Muhammad Hameez Khan	16.6
Yahya Akhrikhar	16.6
Aasif Mohd	16.6

10.4 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
28.04.2026. 20:00	2 Hours	Ibrahim Arda Ilgın Hameez Yahya Aasif	We held a meeting on Google Meet. During the meeting, we discussed the project plan and divided the tasks among all team members. Each member received a clear responsibility. We also discussed what each person needs to do, how the work should be completed, and the expected deadline for each task. The main decision of the meeting was that every member is responsible for completing their assigned task and reporting their progress in the next meeting
02.05.2026. 9:00	8 Hours	Ibrahim Arda Ilgın Hameez Yahya	A meeting was held in the Corvinus Library with the participation of Ibrahim, Arda, Ilgın, Hameez, and Yahya. The purpose of the meeting was to work together on the coding task and discuss the project plan. During the meeting, the team members worked on the coding task together. Ibrahim and Hameez wrote the code, while Yahya, Arda, and Ilgın checked the code for consistency, errors, structure, and other possible problems. The team also discussed the plan and made sure that the completed work followed the project requirements. Any problems found during the checking process were discussed and corrected during the meeting. The result of the meeting was that the coding task was completed and reviewed by the team members.

01.05.2026. 14:00	2 Hours	Aasif	Aasif worked individually on the project. His responsibility was to compile and run the code and prepare the list of files used in the task. During the work, Aasif checked whether the code could compile correctly and run without errors. He also reviewed the project files and prepared a clear list of the required files. The result of this individual work was that the code was tested through compilation and execution, and the list of files for the project was completed.
04.05.2026. 14:00	6 Hours	Yahya Aasif	Yahya and Aasif worked together remotely on the project. The purpose of their work was to test the project, verify the results, debug errors, fix bugs, and finalize the documentation. During the remote work, Yahya and Aasif tested the project against the given requirements. They checked whether the project worked correctly and whether all required functions were completed properly. They also verified the results to make sure the output was correct and consistent. When errors were found, they debugged the project and fixed the bugs. After testing and bug fixing, they reviewed and finalized the project document. The result of this remote work was that the project was tested, verified, corrected, and the final documentation was completed.